

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1206	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	U.Prabhavathi	Reg.No.:	192472067
Department:	CSE-AI	Date:	20-08-26

ANNEXURE A INDUSTRY PROBLEM SOLVING TASK

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

*I **U.Prabhavathi/192472067** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:U.Prabhavathi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

ANNEXURE A – INDUSTRY PROBLEM SOLVING TASK**1. Smartphone Performance Optimization****Problem Statement**

A mobile manufacturer observes lag while switching between applications. The main reason may be inefficient memory access and insufficient cache capacity. When the processor cannot find required data in the cache, it must access DRAM, which has significantly higher latency.

Role of Different Memory Levels

L1 Cache

- L1 is the smallest and fastest cache.
- It is located very close to the CPU cores.
- Stores frequently accessed instructions and data.
- Provides extremely low-latency access.
- Separate instruction and data L1 caches may be used.

L2 Cache

- Larger than L1 but slightly slower.
- Acts as the next level when data is not found in L1.
- Reduces the number of accesses to L3 cache and DRAM.
- Can be dedicated to individual CPU cores.

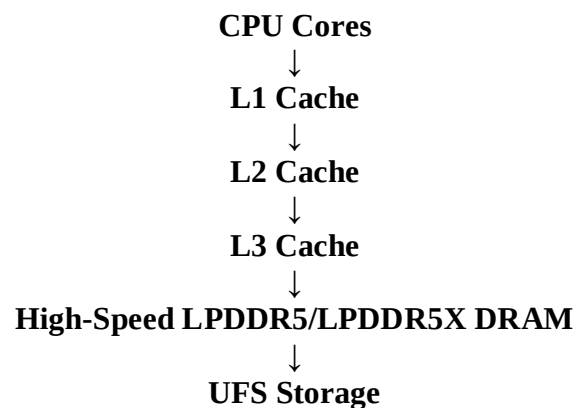
L3 Cache

- Larger than L1 and L2.
- Usually shared among multiple CPU cores.
- Useful for sharing frequently accessed application data.
- Reduces expensive DRAM accesses.

DRAM

- Provides much greater capacity than cache.
- Stores active applications, operating-system data, and background processes.
- It is considerably slower than CPU cache.
- Modern smartphones commonly use low-power LPDDR memory.

Proposed Memory Hierarchy



Proposed Improvements

1. Increase L2/L3 cache capacity where power and chip area allow.
2. Use high-speed LPDDR5/LPDDR5X DRAM.
3. Implement hardware prefetching to predict frequently required data.
4. Improve cache replacement policies.
5. Keep frequently used application data in memory.
6. Reduce unnecessary background applications.

Working

When the user switches to another application, the processor first checks L1 cache. If the required data is unavailable, it checks L2 and then L3. Only after a cache miss at all levels does it access DRAM.

Therefore:

Cache Hit → Very Fast Access

Cache Miss → Next Cache Level → DRAM if necessary

Expected Benefits

- Faster application switching.
- Reduced CPU waiting time.
- Lower memory access latency.
- Better multitasking.
- Improved battery efficiency.
- Improved overall system responsiveness.

Conclusion

A balanced hierarchy consisting of L1, L2, L3 caches and high-speed DRAM can significantly reduce memory access latency. Increasing cache efficiency and using faster low-power DRAM can make smartphone application switching smoother.

2. Cloud Server Memory Bottleneck

Problem Statement

A cloud service provider experiences high latency during database queries. The problem may occur because frequently accessed database information is not available in CPU caches or DRAM, causing repeated accesses to slower memory or storage.

Cache Hierarchy

A CPU cache hierarchy normally consists of:

L1 → L2 → L3 → DRAM

The cache stores frequently accessed information so that the processor does not need to access slower memory repeatedly.

Virtual Memory Paging

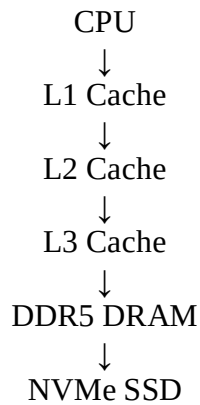
Virtual memory allows the operating system to use storage as an extension of RAM.

When physical RAM becomes insufficient:

DRAM → Page/Swap Storage

Parameter	Cache Hierarchy	Virtual Memory Paging
Main purpose	Reduce CPU memory latency	Extend usable memory
Managed by	Hardware/CPU	Operating system
Fastest level	L1 cache	DRAM
Storage involved	Normally no	Yes
Latency	Extremely low	Much higher when storage is accessed
Best use	Frequently accessed data	Memory overflow
Effect on database	Improves query processing	Can increase latency if paging is excessive

Proposed Architecture



Recommended Improvements

1. Increase DRAM

- More RAM allows more database data to remain in memory.
- Reduces page faults.

2. Use DDR5

- Provides high bandwidth.
- Helps servers process multiple memory requests.

3. Increase Cache Efficiency

- Frequently used database operations benefit from CPU cache.
- Database software should also use appropriate in-memory caching.

4. Reduce Paging

- Allocate enough RAM for database workloads.
- Avoid unnecessary memory pressure.

5. Use NVMe SSD

- If paging or storage access is unavoidable, NVMe provides much faster access than traditional hard drives.

6. Database Optimization

- Use indexes.
- Optimize SQL queries.
- Cache frequently requested results.
- Avoid unnecessary full-table scans.

Working

For a frequently accessed database operation:

CPU → L1 → L2 → L3 → DRAM

If the required information is not available in DRAM and has been moved to storage:

DRAM → NVMe SSD → DRAM → CPU

The second path is much slower.

Expected Benefits

- Reduced database query latency.
- Fewer page faults.
- Better server throughput.
- Higher number of queries per second.
- Better utilization of CPU resources.

Conclusion

Cache hierarchy and virtual memory serve different purposes. Cache improves access speed, while virtual memory provides additional addressable memory. A cloud server should prioritize sufficient DRAM, efficient caching, fast NVMe storage, and optimized database operations.

3. Autonomous Vehicle Data Processing

Problem Statement

An autonomous vehicle continuously receives large amounts of data from:

- Cameras
- LiDAR
- Radar
- Ultrasonic sensors
- GPS
- IMU sensors

This information must be processed in real time. Delays can affect object detection, path planning, and vehicle control.

Memory Comparison

Feature	SRAM	DRAM	MRAM
Speed	Very high	High	High
Latency	Very low	Higher than SRAM	Low
Capacity	Small	Large	Medium
Volatile	Yes	Yes	No
Power	Higher per bit	Moderate	Low standby
Main use	Cache	Working memory	Persistent fast memory

SRAM

SRAM is suitable for:

- CPU cache.
- Frequently accessed sensor data.
- AI accelerator cache.
- Critical real-time processing.

Its major advantage is extremely low latency.

DRAM

DRAM is suitable for:

- Large camera frames.
- LiDAR point clouds.
- AI models.
- Intermediate calculations.
- Sensor fusion data.

Its main advantage is high capacity.

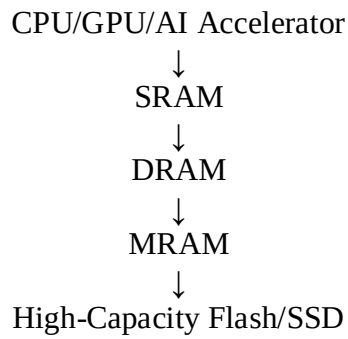
MRAM

MRAM is non-volatile and can retain data without continuous power.

It can be used for:

- Frequently required system data.
- Configuration information.
- Fast persistent storage.
- Critical data that should survive power loss.

Proposed Hierarchy



Example Data Flow

Suppose the vehicle camera detects a pedestrian.

Camera → DRAM → SRAM Cache → AI Processor

Frequently accessed AI parameters and intermediate data can remain close to the processor in SRAM.

Large sensor datasets remain in DRAM.

Important persistent system information can be stored in MRAM.

Improvements

1. Use large SRAM caches for AI processors.
2. Use high-bandwidth DRAM for sensor processing.
3. Use MRAM for fast non-volatile information.
4. Use parallel memory channels.
5. Use DMA to transfer sensor data without excessive CPU involvement.
6. Use memory prefetching.
7. Keep critical real-time data close to the processor.
8. Avoid unnecessary copying of sensor data between memory areas.

Expected Benefits

- Lower processing latency.
- Higher sensor-data bandwidth.
- Faster object detection.
- Better real-time response.
- Lower power consumption.
- Improved system reliability.

Conclusion

A combination of SRAM, DRAM and MRAM provides a good balance between speed, capacity, power consumption and persistence. SRAM handles latency-sensitive operations, DRAM handles large datasets, and MRAM provides fast non-volatile storage.

4. AI Inference Edge Device

Problem Statement

An edge AI device must perform inference locally without depending on cloud servers. It needs to load AI models quickly while maintaining low power consumption.

Examples include:

- Smart cameras.
- Drones.
- IoT devices.
- Industrial monitoring systems.
- Smart healthcare devices.

Memory Comparison

Feature	NAND Flash	DRAM	RRAM
Volatility	Non-volatile	Volatile	Non-volatile
Capacity	Very high	High	Medium
Speed	Lower	High	Very high potential
Power	Low	Moderate	Low

Main purpose Model/storage Active computation Fast model data/in-memory computing

NAND Flash

NAND Flash is useful for:

- Storing AI models.
- Operating system.
- Applications.
- Large datasets.

Its major advantage is high storage capacity and non-volatility.

DRAM

DRAM is used during inference for:

- Loading the active model.
- Intermediate tensors.
- Input images.
- Temporary calculations.

It provides much faster access than NAND Flash.

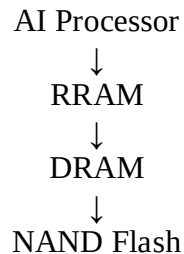
RRAM

RRAM, or Resistive RAM, is a non-volatile memory technology that can potentially support **in-memory** computing.

It can be useful for:

- AI model parameters.
- Frequently accessed weights.
- Reducing movement of data between processor and memory.
- Low-power AI acceleration.

Proposed Hierarchy



Model Loading Process

Initially:

NAND Flash → DRAM → AI Processor

Frequently used parameters can potentially be placed closer to the AI computation hardware.

A more optimized system can use:

NAND Flash → DRAM/RRAM → AI Accelerator

This reduces the amount of data that must repeatedly travel from slower storage.

Recommended Improvements

1. Store complete models in NAND Flash.
2. Load the active model into DRAM.
3. Use RRAM for frequently accessed model parameters where suitable.
4. Compress AI models.
5. Use quantization such as INT8 where accuracy requirements permit.
6. Reduce unnecessary data movement.
7. Use specialized AI accelerators.
8. Use power-efficient memory controllers.

Expected Benefits

- Faster model loading.
- Lower power consumption.
- Reduced data movement.
- Faster inference.
- Longer battery life.
- Better edge-device performance.

Conclusion

The best approach is to use NAND Flash for large permanent storage, DRAM for active model execution, and RRAM for fast non-volatile AI data and potential in-memory computation. This provides a balance between capacity, speed and power efficiency.

5. High-Performance Gaming Console

Problem Statement

A gaming console experiences frame drops when loading large scenes. Modern games contain high-resolution textures, 3D models, audio, lighting information and other large assets.

If the CPU or GPU waits for these assets, rendering performance decreases and frame drops may occur.

L3 Cache

L3 cache:

- Is much faster than DRAM.
- Has smaller capacity.
- Stores frequently accessed CPU data.
- Reduces CPU memory access latency.
- Can improve game logic and CPU-side processing.

DRAM

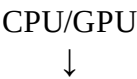
DRAM:

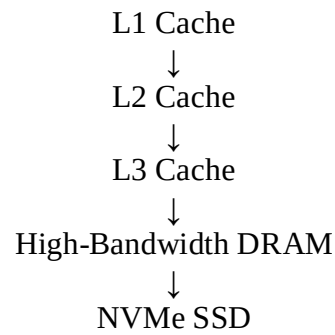
- Has much larger capacity.
- Provides high bandwidth.
- Stores large game assets and working data.
- Is slower than CPU cache but necessary for large workloads.

L3 Cache vs DRAM

Parameter	L3 Cache	DRAM
Capacity	Small	Large
Latency	Very low	Higher
Cost	High	Lower
Bandwidth	High	Very high
Data stored	Frequently used data	Large working datasets
Main advantage	Low latency	Large capacity

Proposed Hierarchy





Scene Loading Example

Consider a large open-world game.

The game may need:

- Character models.
- Buildings.
- Textures.
- Maps.
- Audio.
- Lighting data.

Large assets are stored on the SSD.

Frequently needed assets are transferred into DRAM.

Frequently accessed CPU data is kept in the cache hierarchy.

Therefore:

SSD → DRAM → L3/L2/L1 → CPU/GPU

Recommended Improvements

1. Increase L3 Cache

- Helps reduce CPU accesses to DRAM.
- Useful for frequently accessed game logic and data.

2. Use High-Bandwidth DRAM

- Allows large amounts of graphics and game data to be transferred quickly.
- Helps keep the GPU supplied with data.

3. Use Fast NVMe SSD

- Reduces initial scene loading time.
- Enables faster streaming of game assets.

4. Prefetching

- Predict which game assets will be required next.

- Load them into DRAM before they are needed.

5. Asset Streaming

- Load only the game assets required for the current scene.
- Avoid loading the entire game world into memory.

6. Data Compression

- Compressed textures and other assets reduce storage and memory bandwidth requirements.

Expected Benefits

- Reduced frame drops.
- Faster scene loading.
- Smooth gameplay.
- Better GPU utilization.
- Faster asset streaming.
- Improved overall gaming performance.

Conclusion

L3 cache and DRAM perform different roles and should not be considered replacements for each other. L3 cache minimizes latency for frequently used data, while high-bandwidth DRAM provides the capacity and bandwidth required for large game workloads. Combining these with a fast NVMe SSD and intelligent asset streaming provides an optimized gaming memory hierarchy.